Mouse, the Language

URL: http://primepuzzle.com/mouse/mouse5.html

Last changed on Tuesday, April 24, 2007

Our final sample program introduces recursive macros (ones that call themselves) and one additional arithmetic operator, \, which pops two values from Stack and pushes the remainder when the first is divided by the second.

It asks for a number and then prints out the prime factors of your number. If you don't know what a prime number is, we're in a little bit of trouble already but, basically, numbers are prime when they can't be written as the product of two different numbers. So, 51 isn't prime because 51 = 3 * 17. 101 is prime (try to find a number that goes into it evenly).

I might mention here that Mouse can only handle small, integer numbers. Thus the requirement that the number you enter be less than 32758.

The code below is fairly well commented so if you do look into this, read the comments. Additional remarks are given following the code.

```
~ prime.mse - lrb - 2/15/2007,3/24/2007

("!Enter a number in the range [2,32757] : "?Q:1Q.<Q.32758<*0=^)

"!Do you want to check "
"1. this number only or 2. the range [" Q.!","Q.10+!"]!"
"!1 or 2 ? "?'o:"!"

o.'1=["!"Q.!" : "#G,Q.,2;]
o.'2=[Q.1-l:Q.10+Q:(Q.l.-^"!"Q.!" : "#G,Q.,2;Q.1-Q:)]

"!!Strike any key ... "?'

~ find a prime factor of 1% using 2% as the initial candidate

$G 1%n:2%f:
~ 2 is special; if it's a candidate and not a divisor, start w/ 3
2%2=[n.2\[3f:]]
(n.f.\^f.2+f:) ~ f is now the smallest factor of n
~ next line figures out how often f goes into n
1c: n.f./d: f.g: (d.f.\0= ^ 1c.+c: d.f./d: g.f.*g:)
~ print it (possibly with an exponent) followed by a space
f.!c.1>['^!'c.!]" "
```

```
g.n.<[#G,d.,f.;] ~ call self if not done
f.Q.=["is prime."]@
```

Let's face it. This is pretty complicated stuff! Notice how the G macro calls itself. All the variables the G macro uses are what are known as local variables (except for Q, which is a global). G works correctly because the Mouse interpreter creates a new set of local variables at every macro call.

Here's an explanation of the line

```
1c: n.f./d: f.g: (d.f.\0= ^ 1c.+c: d.f./d: g.f.*g:)
```

Set c to 1 (c is for count).

Load d with n divided by f (because when this line gets executed we know n is divisible by f).

Load g with f (g will end up holding the value f^c when the loop is exited and we'll compare this to n to see if we need to call the macro again). Loop while d is still divisible by f (which it might not be at all), bumping c up by 1 each time and recomputing d and g.

When you get out of all this you have the exponent the factor f should have in the prime factorization of your number.

Here's a screen shot of a sample run.

```
B0>mouse-p
MOUSE, Peter Grogono, 1983; Lee Bradley, 2007
Enter Mouse program's name : prime.mse

Enter a number in the range [2,32757] : 96

Do you want to check 1. this number only or 2. the range [96,106]

1 or 2 ? 2

106 : 2 53
105 : 3 5 7
104 : 2^3 13
103 : 103 is prime.
102 : 2 3 17
101 : 101 is prime.
100 : 2^2 5^2
99 : 3^2 11
98 : 2 7^2
97 : 97 is prime.
96 : 2^5 3

Strike any key ...
```

B0>

0 - Emulator, Interpreter and the classic Hello, World! program
1 - Output string, output character, new line, Mouse's Stack, push character, end program
2 - Loops, variables, assignment, evaluation, comparison, comments, arithmetic, input number, output number
3 - Introduction to defining and calling macros
4 - If, input character, more on macros
5 - Recursive macros, the remainder operator, the small, whole number limitation of Mouse
6 - A few remarks about CP/M and MYZ80
7 - Testing and the trace feature, contact information